A Deep Learning Approach for Cross-Platform Malware Detection & Classification based on Image Representation of Byte code



Presented by
Misnad Khasim
Reg. 24030272

Supervised by
Dr. Vinod P
Professor, DCA
CUSAT

17 Apr 2026

Table of Contents

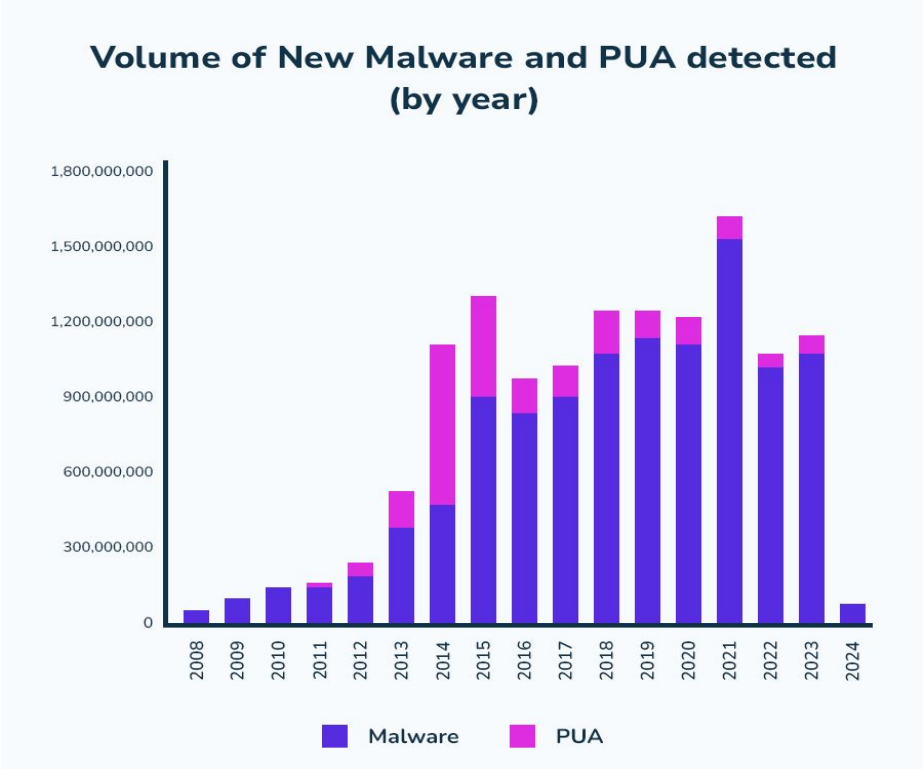
Introduction	Problem statement
	Scope of Work
	Objective
Related Works / Literature Review	
Methodology	Architecture
	Binary to Image Conversion
	Segmentation
	Ensemble Model
Experiment	Dataset
	Technologies Used
	Evaluation Metric
Results / Evaluation	Performance
Discussion / Limitations	
Conclusions / Future Work	

Introduction

The Escalating Scale of the Malware Threat

- Over **450,000** new malware samples are registered daily. (Source: cvedetails.com)
- Mobile malware increased by 15% between the first and second quarters of 2020. (Source: McAfee 2020)
- The sheer volume of samples renders manual assessment impossible. Furthermore, malware is becoming **increasingly sophisticated**, posing severe threats to privacy and security. (Sources: Google 2018; Malwarebytes Lab 2020; Kaspersky Lab 2017)
- While automated detection approaches exist, detecting all malware remains an "unattainable dream." (Sources: Kang et al. 2015; Petsas et al. 2014; Zheng et al. 2013; Faruki et al. 2013)
- Traditional **signature-based detection is too slow** and fails against zero-day attacks.
- The main mode of malware attack is through **program execution**. (Source: http://www.cvedetails.com/vulnerabilities-by-types.html)

The Escalating Scale of the Malware Threat



Source: https://app.stationx.net/articles/malware-statistics

Limitation of Traditional approaches

- Traditional ML approaches—whether static, dynamic, or hybrid, **rely entirely on manual feature engineering** to approximate app behaviour. (Sources: Arp et al. 2014; Garcia et al. 2018; Onwuzurike et al. 2019; Fereidooni et al. 2016; Cai et al. 2019; Wu and Hung 2014; Martinelli et al. 2017)
- Hand-crafted features often **miss critical information** necessary to distinguish malicious from benign apps. Good feature engineering remains an open problem.
- The community is **shifting toward representing programs as images** to leverage the massive performance gains seen in **computer vision**.
- **Microsoft and Intel Labs** successfully demonstrated this with **STAMINA**, a Deep Learning approach that detects malware using image representations of binaries via transfer learning.

Current Image based approaches

- Current approaches often jump straight to **non-trivial representations** (like color) and unnecessarily complex architectures like Inception-v3. (Source: Szegedy et al. 2016)
- Converting sequential DEX bytecode into 2D square or rectangular images **destroys natural locality** and creates "**artificial locality**" (e.g., false relationships formed when a bytecode sequence breaks at the end of a row and continues on the next).
- Architectural Mismatch caused by using 2D Convolutional Neural Networks (CNNs) on these artificial rectangles is fundamentally flawed, as there is no actual relationship between pixels in the same column.

Objective

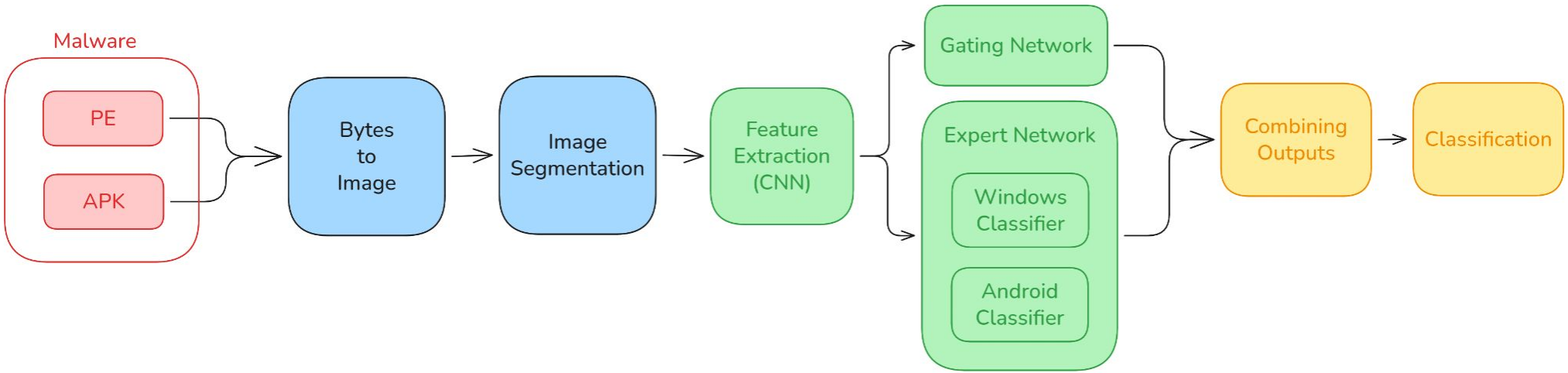
- A straightforward 1D "vector" image-based approach.
- By combining a simple image representation with a basic neural network architecture, this approach avoids 2D distortion and establishes a minimal-performance baseline for the research community to build upon.

Related Works

Evolution Stage	Approach / Methodology	Key Characteristics	Primary Advantages	Core Limitations	Key Citations
1. The Advent of 2D	2D Image Grids	Converts raw byte streams into fixed-width 2D matrices. Analysed using standard 2D CNNs and transfer learning.	Bypasses manual feature engineering entirely. Leverages highly mature computer vision models.	Creates "artificial locality" by fracturing continuous sequential logic at the right-hand edge of the image grid.	Nataraj et al. (2011) Chen et al. (2020) [STAMINA]
2. The Shift to 1D	1D "Vector" Images	Maps byte streams to continuous 1D greyscale arrays. Analysed using 1D CNNs.	Preserves true structural byte integrity. Eliminates quantization noise and spatial distortion introduced by 2D reshaping.	Dealing with extremely long sequence lengths can be computationally heavier than processing compacted 2D grids.	Daoudi et al. (2021) [DEXRAY] Various Authors (2024) [Signal-Based 1D]
3. Hierarchical & Routing	Mixture of Experts (MoE) & Ensembles	Employs gating networks to route multi-platform binaries to specialized, platform-specific 1D experts.	Effectively handles OS heterogeneity. Minimizes "negative transfer" (conflicting patterns) between fundamentally different file types (e.g., Android DEX vs. Windows PE).	Introduces higher architectural complexity and requires sophisticated hyperparameter tuning for the routing logic.	Kumar et al. (2024)

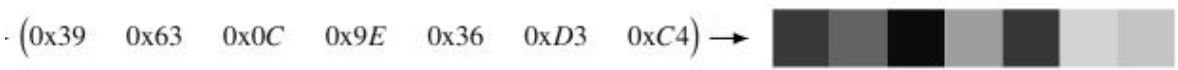
Methodology

Data Pipeline



Bytes as Ints

- Every file is just a string of bytes.
- Each byte value is mapped into pixel intensity
  (0 = Black, 255 = White).
- Different sections of a binary create distinct visual textures.
- Malware families share code structures, resulting in similar visual fingerprints that Computer Vision can detect.



APK to Images

- Android executable instructions are compiled into **Dalvik Executable** (classes.dex) files.
- Larger APK archives frequently contain **multiple** DEX files (e.g., classes.dex, classes2.dex).
- APKs are **unpacked**, and all extracted DEX files are **sequentially concatenated** to form a single, unified 1D byte-image.

Input: APK file
Output: 8-bit grey-scale "vector" image of size

```
bytestream ← ∅
for dexFile in APK do
    bytestream ← bytestream +
     dexFile.toByteStream()
end
l ← bytestream.length()
img ← generate8bitGreyScaleVectorImage(bytestream,
 l)
img.save()
```

2d Images

- Selected a traditional 2D CNN architecture (similar to DexRay) to serve as our primary comparison model.
- Bytes are tuned into images of equal width and height.
- Padding is added if necessary.
- The images are resized to **128x128**.

Input: APK file
Output: 8-bit grey-scale "square" image of size (128, 128)

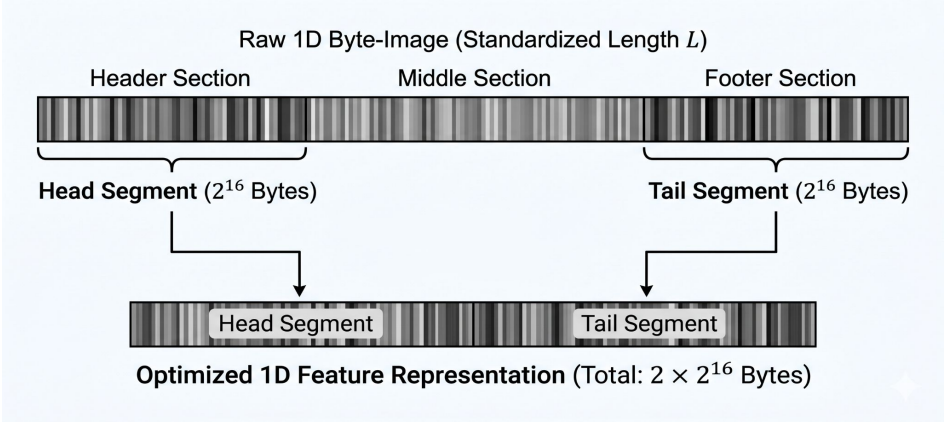
```
bytestream ← ∅
for dexFile in APK do
    bytestream ← bytestream +
      dexFile.toByteStream()
end
l ← bytestream.length()
sqrt ← ⌈ √l ⌉
sq ← sqrt^2
while bytestream.length() ≠ sq do
    bytestream ← bytestream + "\x00" // padding with
      zeros
end
// At this point, bytestream is divided in sqrt part of
  length sqrt
// In other words, it is represented as a sqrt × sqrt
  matrix
img ←
  generate8bitGreyScaleSquareImage(bytestream, sqrt)
img.resize_to_size(height=128, width=128)
img.save()
```

Segmenting the Images

- Partitioned the continuous 1D byte-images to identify which specific regions of the binary contain the **highest density** of malicious signatures.
- Systematically evaluated different extraction locations (head, middle, and tail of the file) across **multiple segment window** sizes (e.g., **2^{12}, 2^{14}, 2^{16}, and 2^{18}** bytes).
- Testing revealed that middle sections yielded lower predictive value, while the absolute **beginning** (headers/metadata) and absolute **end** (footers/appended payloads) contained the most **critical features**.

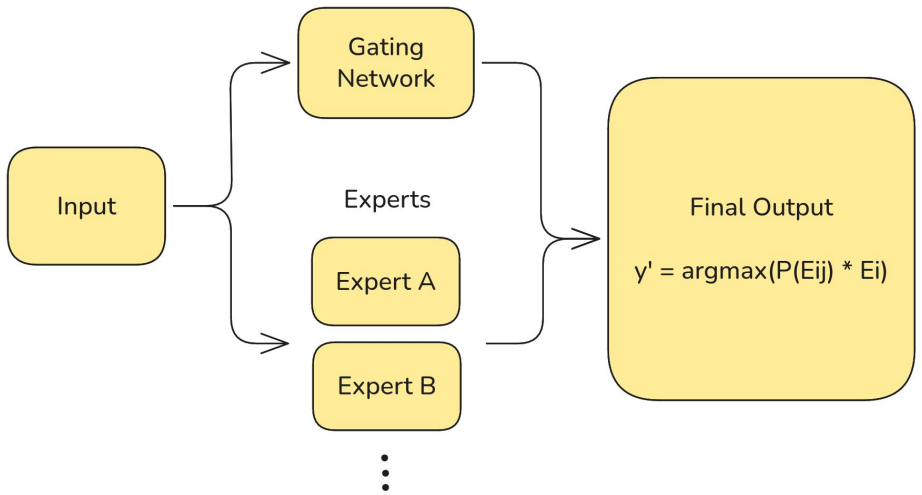
Segmenting the Images

- Empirical results demonstrated that extracting **exactly 2^{16} (65,536)** bytes from a given region maximized classification accuracy without introducing excessive noise.
- The final model architecture explicitly concatenates the first 2^{16} bytes and the last 2^{16} bytes to form a single, highly-optimized representation for final prediction.



Expert Models and Routing

- Deployed **two independent 1D CNN models** trained exclusively on Windows binaries and the other on Android binaries. This isolates platform-specific feature extraction and prevents "**negative transfer**" (where Windows byte patterns might confuse an Android-focused classifier, and vice versa).
- A **separate DNN** processes the unified 1D input image to determine the structural origin of the binary. It outputs a **routing probability** distribution indicating how likely the input is a Windows or Android file.



Expert Models and Routing

- **Expert Weighting**: Rather than rigidly hard-coding a binary down a single path, the gating network assigns a confidence weight [0,1] to each expert's output.
- **Final Output**: The final classification is calculated by multiplying the gating network's probabilities by each expert's respective classification output, summing those values, and selecting the class with the highest overall score.

$$y = \sum_{i \in \{\text{Windows}, \text{Android}\}} G_i(x) \cdot E_i(x)$$

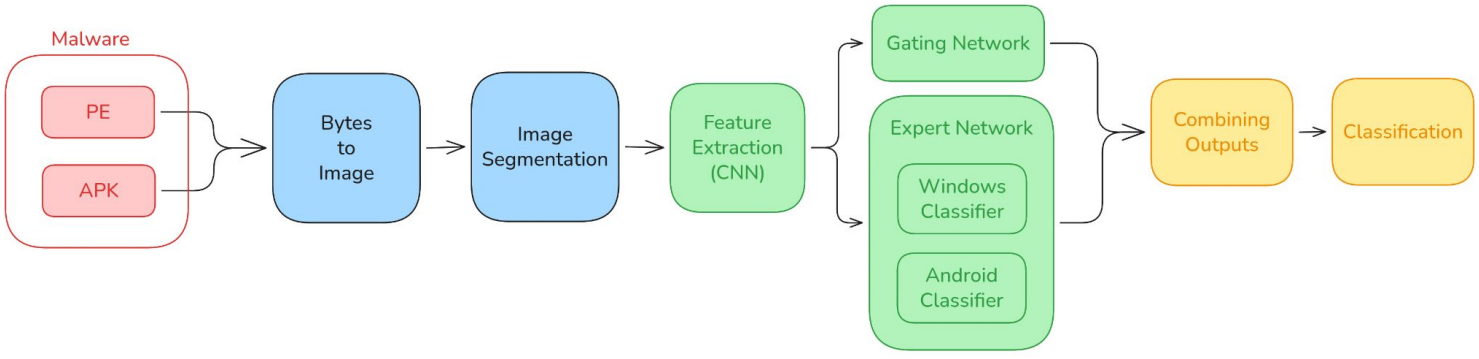
$$\hat{y} = \arg\max(y)$$

Model Architecture

- **Feature Extraction Module**
  - Architecture: Standard Convolutional Neural Network (CNN)
  - Layer Sequence: Convolution → Pooling → Convolution → Pooling → Convolution → Pooling → Dropout → Flatten
  - Function: Acts as the automated feature extractor, sliding over the byte-image to identify local malicious patterns and condensing them into a flattened, high-density feature vector.
- **The Gating Network**
  - Architecture: 2-Layer Multi-Layer Perceptron (MLP)
  - Layer Sequence: Extracted Features Input → Linear Transformation → Logits Output
  - Function: Takesthe flattened feature vector and calculates the routing probabilities to determine which expert (Windows or Android) is best suited for the classification.

Model Architecture

- **The Expert Networks**
  - Architecture: 3-Layer Multi-Layer Perceptron (MLP)
  - Layer Sequence: Features → Linear → Normalization → Dropout → Linear → Normalization → Dropout → Logits
  - Function: Processes the features through a deep pathway to output the final classification decision.



GradCAM and Explainability

- Evaluated our **1D CNN** experts against an identically structured **2D CNN** baseline to isolate and quantify the performance benefits of 1-dimensional sequence processing.
- We also used hyperparameter optimizer to find the optimal values.
- Implemented 1D Grad-CAM (**Gradient-weighted Class Activation Mapping**) to introduce transparency into the model's decision-making process.
- Used Grad-CAM activation maps to visually isolate the specific byte sequences and regions within the 1D image that most heavily triggered the malicious classification.
- Used the Grad-CAM heatmaps to empirically validate our segmentation strategy, confirming that the highest predictive signals reside in the header and footer regions.

Experiment

Dataset Overview & Class Distribution

- **Android Dataset (CICMalDroid 2020)**
  - Total Samples: **11,583 files**
  - Total Classes: **5**
  - Categories:
    - Malicious: ***Adware, Banking, Riskware, SMS***
    - Safe: ***Benign***
- **Windows Dataset (Malhub)**
  - Total Samples: **26,411 files**
  - Total Classes: **20**
  - Categories: ***adload, agent, alureon, bho, ceeinject, cycbot, delfinject, fakerean, hotbar, lolyda, obfuscator, onlinegames, rbot, renos, startpage, vobfus, vundo, winwebsec, zbot, zeroaccess***

Class Imbalance

Android

Class Name	Sample Count
Sms_final	3,902
Riskware_final	2,546
Banking_final	2,087
Benign_final	1,795
Adware_final	1,253

Windows

Class Name	Sample Count	Class Name	Sample Count
winwebsec	3,651	startpage	1,136
vundo	1,793	zeroaccess	1,119
zbot	1,785	fakerean	1,063
hotbar	1,491	adload	1,050
obfuscator	1,445	cycbot	1,029
alureon	1,328	rbot	1,017
renos	1,312	vobfus	926
onlinegames	1,293	lolyda	915
bho	1,176	ceeinject	894
delfinject	1,146	agent	842

Technology Stack & Implementation

- **High-Performance Preprocessing: Rust (rust-script)**
  - A custom, memory-safe script to rapidly parse raw Windows and Android binaries and transform them into standardized 1D byte-images.
- **Deep Learning Framework: PyTorch**
  - Built, trained, and evaluated the 1D CNN experts, the dynamic gating network, and the overarching Mixture of Experts (MoE) architecture.
- **Automated Optimization: Optuna**
  - Conducted systematic hyperparameter tuning to optimize CNN kernel sizes, filter depths, and routing parameters efficiently.
- **Data Visualization & Interpretability: Matplotlib & Seaborn**
  - Plotted training/validation metrics and generated the Grad-CAM heatmaps to visually explain the model's byte-level focus.
- **Development Environment: Jupyter Notebook**
  - For interactive prototyping and side-by-side comparisons of the 1D vs. 2D baselines.

Hyperparameters

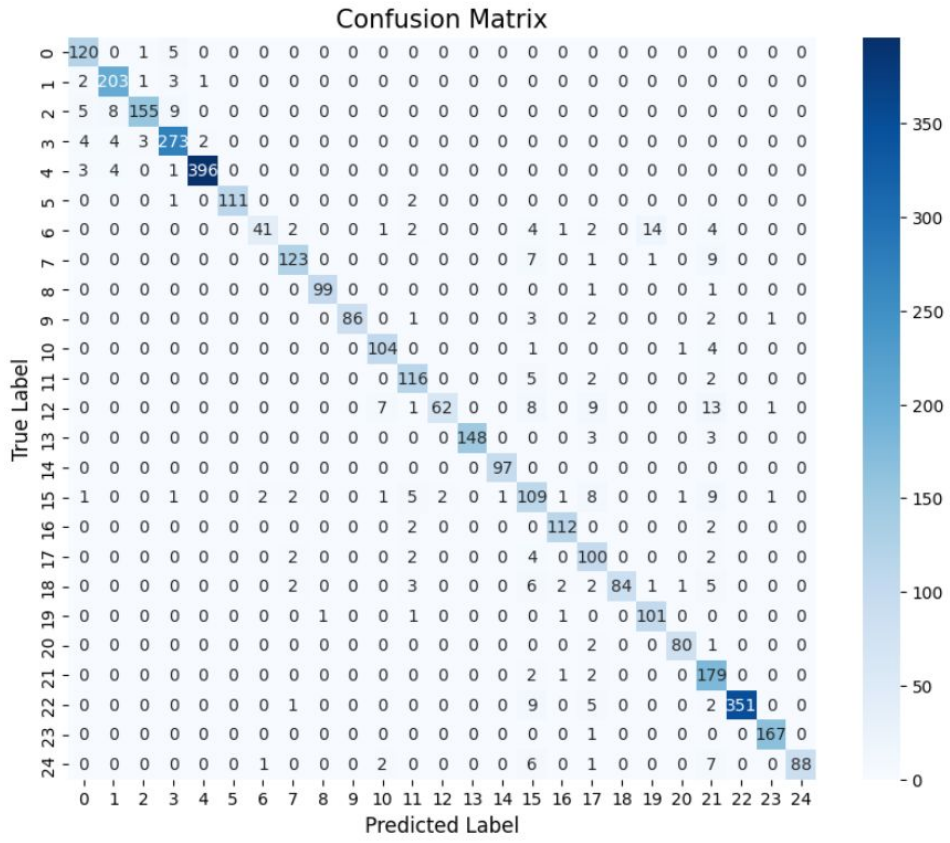
- **Optimizer**: Adam with weight decay of 1e-5
- **Loss**: Negative log likelihood loss
- **Epoch**: 30
- **Train**, **Val, Test Split**: 80, 10, 10
- **Learning** Rate: 5e-3

Weighted Avg F1 Score

- To minimizes the risk of "False Negatives" (allowing malware to execute) without creating excessive "False Positives" (blocking safe executables) we chose **Weighted F1-Score (Harmonic Mean of Precision & Recall)** as the deciding metric.

Results / Evaluation

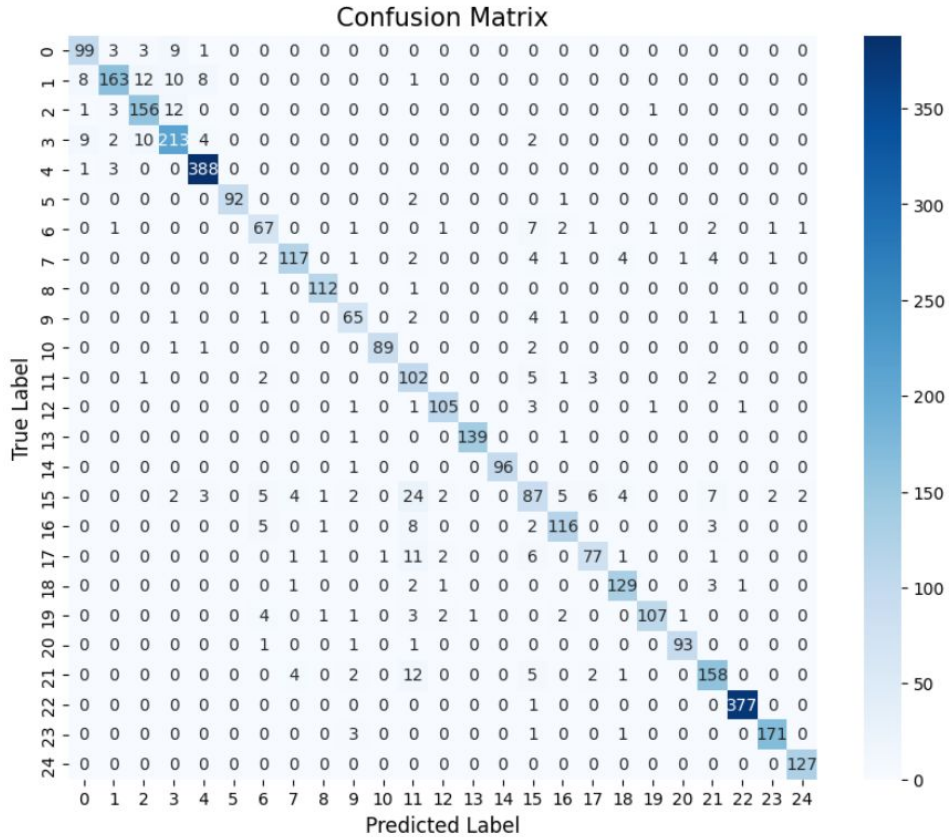
1D Classification Results



----------Classification Report----------

	precision	recall	f1-score	support
Android_Adware_final	0.8889	0.9524	0.9195	126
Android_Banking_final	0.9269	0.9667	0.9464	210
Android_Benign_final	0.9688	0.8757	0.9199	177
Android_Riskware_final	0.9317	0.9545	0.9430	286
Android_Sms_final	0.9925	0.9802	0.9863	404
Windows_adload	1.0000	0.9737	0.9867	114
Windows_agent	0.9318	0.5775	0.7130	71
Windows_alureon	0.9318	0.8723	0.9011	141
Windows_bho	0.9900	0.9802	0.9851	101
Windows_ceeinject	1.0000	0.9053	0.9503	95
Windows_cycbot	0.9043	0.9455	0.9244	110
Windows_delfinject	0.8593	0.9280	0.8923	125
Windows_fakerean	0.9688	0.6139	0.7515	101
Windows_hotbar	1.0000	0.9610	0.9801	154
Windows_lolyda	0.9898	1.0000	0.9949	97
Windows_obfuscator	0.6646	0.7569	0.7078	144
Windows_onlinegames	0.9492	0.9655	0.9573	116
Windows_rbot	0.7092	0.9091	0.7968	110
Windows_renos	1.0000	0.7925	0.8842	106
Windows_startpage	0.8632	0.9712	0.9140	104
Windows_vobfus	0.9639	0.9639	0.9639	83
Windows_vundo	0.7306	0.9728	0.8345	184
Windows_winwebsec	1.0000	0.9538	0.9764	368
Windows_zbot	0.9824	0.9940	0.9882	168
Windows_zeroaccess	1.0000	0.8381	0.9119	105
accuracy			0.9224	3800
macro avg	0.9259	0.9042	0.9092	3800
weighted avg	0.9313	0.9224	0.9227	3800

2D Classification Results



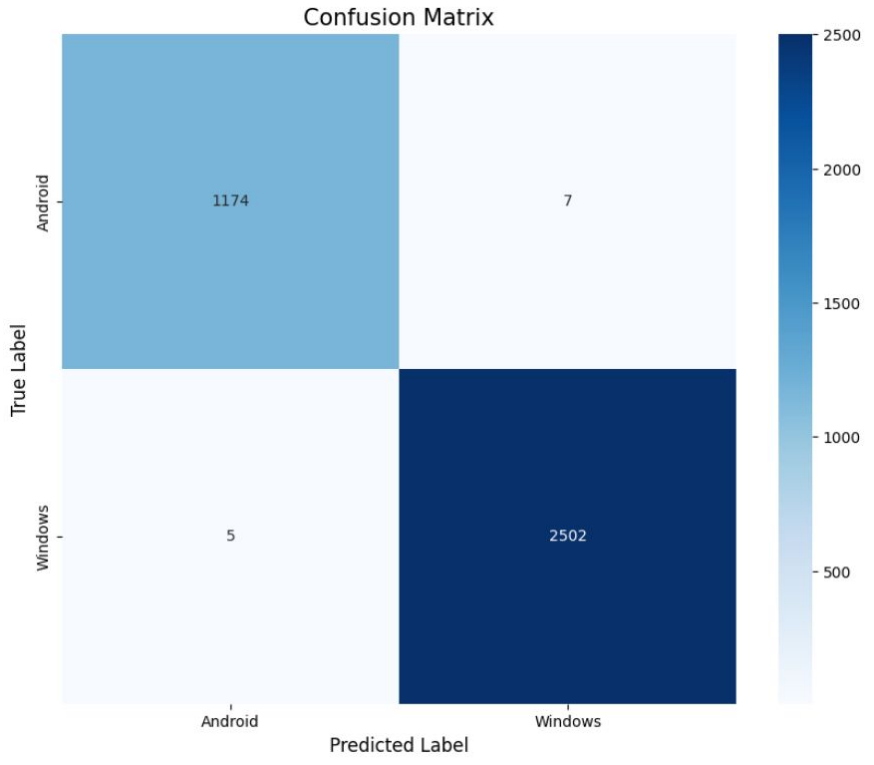
----------Classification Report----------

	precision	recall	f1-score	support
Android_Adware_final	0.8390	0.8609	0.8498	115
Android_Banking_final	0.9314	0.8069	0.8647	202
Android_Benign_final	0.8571	0.9017	0.8789	173
Android_Riskware_final	0.8589	0.8875	0.8730	240
Android_Sms_final	0.9580	0.9898	0.9737	392
Windows_adload	1.0000	0.9684	0.9840	95
Windows_agent	0.7614	0.7882	0.7746	85
Windows_alureon	0.9213	0.8540	0.8864	137
Windows_bho	0.9655	0.9825	0.9739	114
Windows_ceeinject	0.8228	0.8553	0.8387	76
Windows_cycbot	0.9889	0.9570	0.9727	93
Windows_delfinject	0.5930	0.8793	0.7083	116
Windows_fakerean	0.9292	0.9375	0.9333	112
Windows_hotbar	0.9929	0.9858	0.9893	141
Windows_lolyda	1.0000	0.9897	0.9948	97
Windows_obfuscator	0.6744	0.5577	0.6105	156
Windows_onlinegames	0.8923	0.8593	0.8755	135
Windows_rbot	0.8652	0.7624	0.8105	101
Windows_renos	0.9214	0.9416	0.9314	137
Windows_startpage	0.9727	0.8770	0.9224	122
Windows_vobfus	0.9789	0.9688	0.9738	96
Windows_vundo	0.8729	0.8587	0.8658	184
Windows_winwebsec	0.9921	0.9974	0.9947	378
Windows_zbot	0.9771	0.9716	0.9744	176
Windows_zeroaccess	0.9769	1.0000	0.9883	127
accuracy			0.9066	3800
macro avg	0.9017	0.8976	0.8977	3800
weighted avg	0.9101	0.9066	0.9067	3800

Gating Network

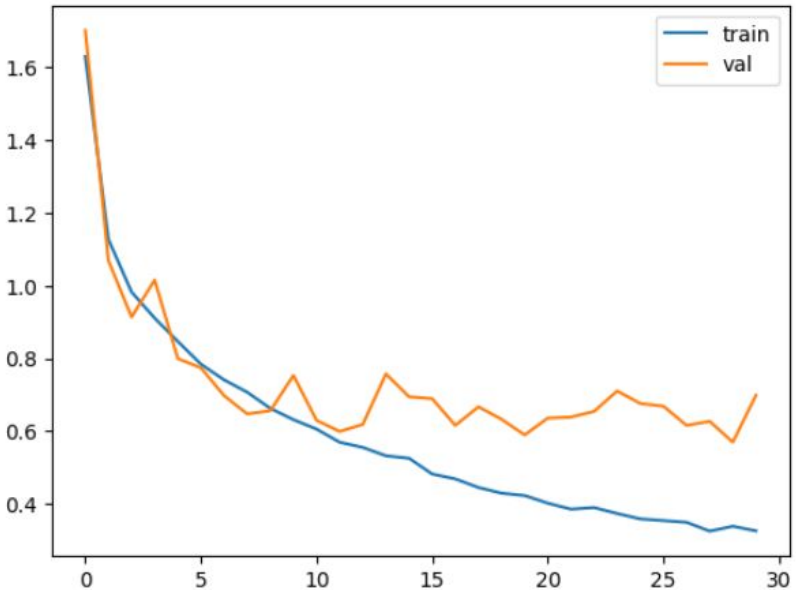
----------Classification Report----------

	precision	recall	f1-score	support
Android	0.9958	0.9941	0.9949	1181
Windows	0.9972	0.9980	0.9976	2507
accuracy			0.9967	3688
macro avg	0.9965	0.9960	0.9963	3688
weighted avg	0.9967	0.9967	0.9967	3688

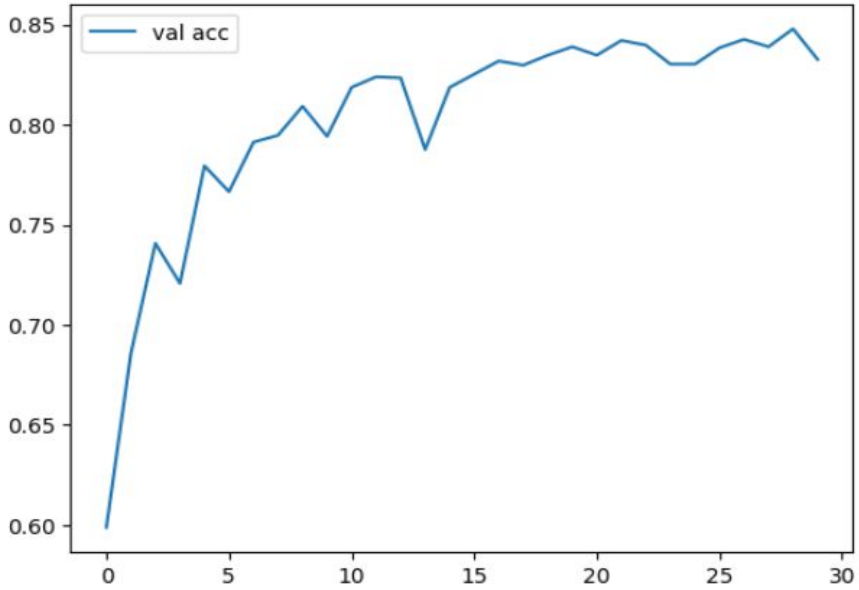


Training curves (30 epochs)

Training and Validation Loss

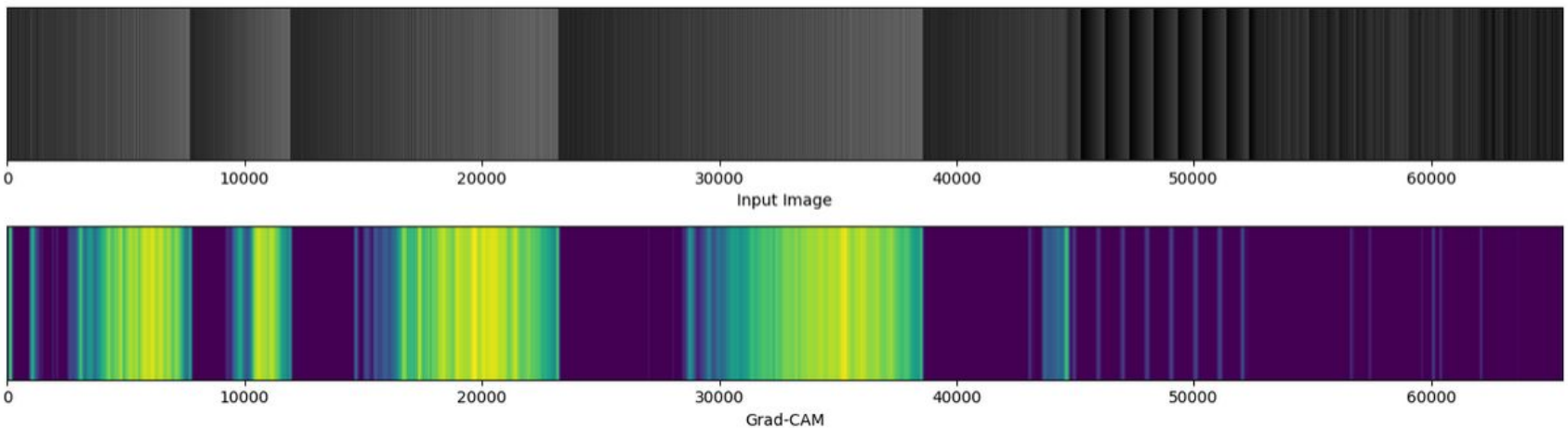


Validation Accuracy



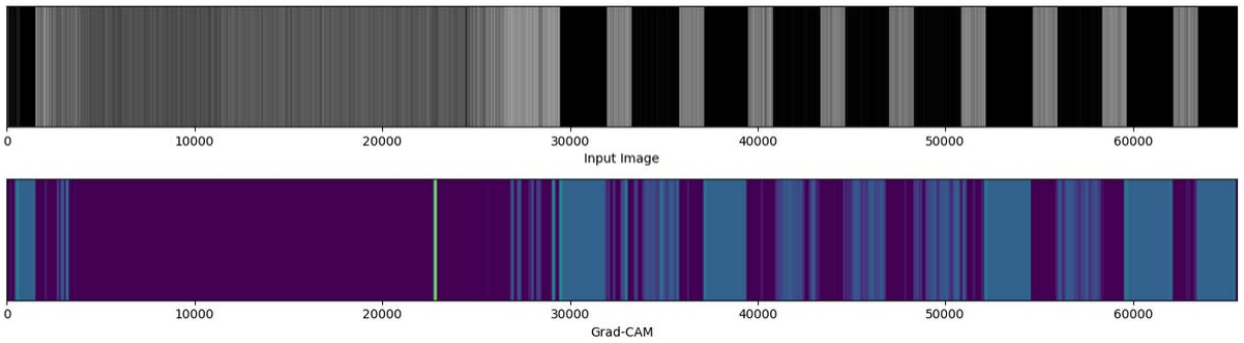
GradCAM

Android
Banking_final

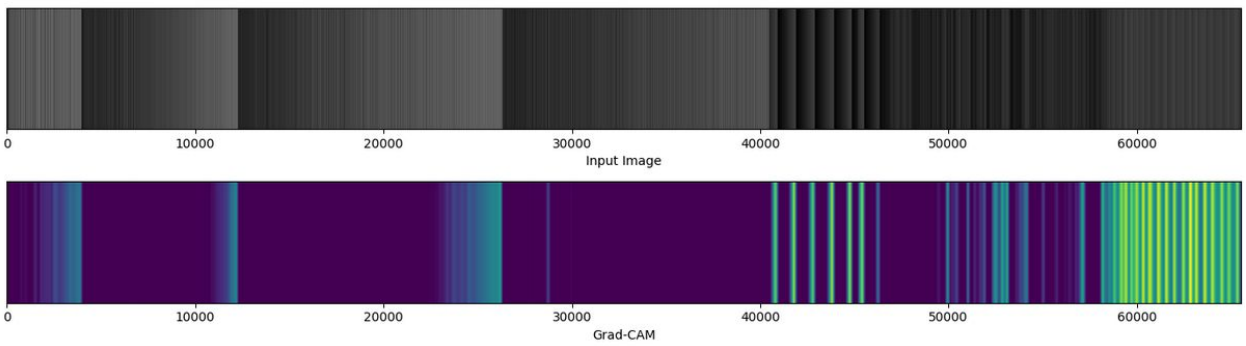


GradCAM

Windows renos



Android Adware_final



Discussions / Limitations

Key Takeaways

- **1D Superiority**: Validating the DexRay hypothesis, our 1D sequence processing preserves true bytecode integrity and outperforms 2D models by eliminating "artificial spatial locality."
- **Efficiency**: Similar to recent stacked ensemble research, our selective extraction of head/tail segments (2×2^{16} bytes) drastically reduces computational overhead while maintaining high signal density.
- **MoE Advantage**: The Hierarchical MoE successfully isolates platform-specific features, proving that a gating network effectively prevents "negative transfer" between Android and Windows binaries.

Limitations

- **Obfuscation Vulnerability**: As a purely static, byte-level analyzer, the model remains vulnerable to advanced packing, polymorphism, or string encryption that masks the underlying code structure.
- **Rigid Segmentation**: While optimal for our dataset, permanently discarding the "middle" bytes may cause the model to miss fragmented payloads hidden deep within unusually large executables.
- **Lack of Dynamic Context**: The architecture evaluates static artifacts but cannot analyze runtime behaviors (e.g., memory injection, API calls, or network traffic).

Conclusions / Future Work

Conclusion

- **Architectural Milestone**: Successfully engineered and validated a Hierarchical Mixture of Experts (MoE) capable of cross-platform malware classification without negative transfer.
- **1D Superiority Proven**: Conclusively demonstrated that preserving 1D sequence integrity (F1: 0.9227) outperforms 2D spatial reshaping (F1: 0.9067) by maintaining natural structural byte locality.
- **Optimized Pipeline**: Make an extremely fast parser and used a simple convolution model and yet maximizes signal-to-noise ratio with minimal computational overhead.

What more can be done

- **Adversarial Resilience**: Stress-testing the 1D CNN experts against advanced packing mechanisms, polymorphism, and structural encryption to evaluate robustness.
- **Dynamic Behavioral Integration**: Evolving the static byte-level architecture into a hybrid model by integrating a dynamic analysis module (e.g., evaluating runtime API calls or memory behavior).
- **Expanding the Domains**: Adding GNU/Linux, iOS, PDF malwares and training experts for each.
- **Production Scaling & Publication**: Expanding the dataset to include highly imbalanced, real-world zero-day samples and optimizing the Rust-to-PyTorch pipeline for real-time, production-grade inference.

References

Key References (1)

[1] T. Akiba, S. Sano, T. Yanase, T. Ohta, and M. Koyama, "Optuna: A next-generation hyperparameter optimization framework," in Proc. 25th ACM SIGKDD Int. Conf. Knowl. Discovery Data Mining, 2019, pp. 2623–2631.

[2] J. Chen et al., "STAMINA: Scalable deep learning approach for malware classification," arXiv preprint arXiv:2003.04439, 2020.

[3] N. Daoudi et al., "DEXRAY: A simple, yet effective deep learning approach to Android malware detection based on image representation of bytecode," arXiv preprint arXiv:2109.03326, 2021.

[4] R. A. Jacobs, M. I. Jordan, A. S. Nowlan, and G. E. Hinton, "Adaptive mixtures of local experts," Neural Computation, vol. 3, no. 1, pp. 79–87, 1991.

[5] K. Kumar, H. L. Mandoria, and C. S. Negi, "Efficient malware classification using 1D-CNN and stacked ensemble techniques," in 2024 IEEE Int. Conf. Smart Power Control Renewable Energy (ICSPCRE), 2024.

[6] S. Kumar and A. Kumar, "MalSFF: Multi-architecture malware detection using multi-static feature fusion based on image visualization and learning methods," Computers and Electrical Engineering, vol. 129, p. 110834, 2026.

Key References (2)

[7] L. Kurlandski, H. Berger, Y. Pan, and M. Wright, "Beyond raw bytes: Towards large malware language models," in Proc. Network and Distributed System Security Symp. (NDSS), 2026.

[8] S. Mahdavifar et al., "CICMalDroid 2020: Android Malware Dataset," Canadian Institute for Cybersecurity, 2020.

[9] Malhub, "Malhub: Open-source malware repository." [Online]. Available: GitHub / Community Repository.

[10] L. Nataraj, S. Karthikeyan, G. Jacob, and B. S. Manjunath, "Malware images: visualization and automatic classification," in Proc. 8th Int. Symp. Visualization for Cyber Security, 2011, pp. 1–7.

[11] R. R. Selvaraju, M. Cogswell, A. Das, R. Vedantam, D. Parikh, and D. Batra, "Grad-CAM: Visual explanations from deep networks via gradient-based localization," in Proc. IEEE Int. Conf. Comput. Vis. (ICCV), 2017, pp. 618–626.

[12] N. Shazeer et al., "Outrageously large neural networks: The sparsely-gated mixture-of-experts layer," in Int. Conf. Learning Representations (ICLR), 2017.

[13] G. Smith, "+65 Malware Statistics for 2025," StationX, Dec. 2024. [Online]. Available: https://www.stationx.net/malware-statistics/

Thank You